

HDOC – Experiment 30

Question

Write a C program to perform the following menu-driven operations on a given array:

1. Sort the elements using Bubble Sort algorithm and perform a Binary search.
2. Perform Insertion and Deletion in a given valid position.
3. Rotate the array elements to the right by k positions.
4. Exit

❖ Algorithm

Step 1: Start.

Step 2: Read the number of elements n.

Step 3: Validate n. If $n \leq 0$, ask the user to enter it again.

Step 4: Read n elements into the array.

Step 5: Display the menu repeatedly:

1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation
4. Exit

Step 6: Read the user's choice.

Choice 1: Bubble Sort and Binary Search

1. Sort the array in ascending order using Bubble Sort.
2. Display the sorted array.
3. Read the element to be searched.
4. Set low = 0 and high = n - 1.
5. Repeat while low <= high:
 - Calculate mid = (low + high) / 2.
 - If a[mid] equals the search element, report its position.
 - If the search element is smaller than a[mid], set high = mid - 1.
 - Otherwise, set low = mid + 1.
6. If no match is found, display "Element not found".

Choice 2: Insertion and Deletion

Ask the user whether to perform:

1. Insertion
2. Deletion

For insertion:

1. Read the position and new element.
2. Position must be from 1 to n + 1.
3. Shift elements one position to the right.
4. Insert the new element.
5. Increase n by 1.

For deletion:

1. Read the position.
2. Position must be from 1 to n.
3. Store/display the element being deleted.
4. Shift subsequent elements one position to the left.

5. Decrease n by 1.

Choice 3: Right Rotation

1. Read k.

2. Reduce unnecessary rotations using $k = k \% n$.

3. Repeat k times:

- Store the last element.
- Shift all remaining elements one position to the right.
- Put the stored element at index 0.

4. Display the rotated array.

Choice 4

Terminate the program.

Step 7: For any other menu choice, display an appropriate error message.

Step 8: Stop.

❖ Program in C

```
#include <stdio.h>
```

```
#define MAX 100
```

```
int main()
```

```
{
```

```
    int a[MAX];
```

```
    int n, i, j;
```

```
int choice, subChoice;  
int temp, key;  
int low, high, mid, found;  
int position, element;  
int k, r, last;
```

```
/* Read and validate number of elements */
```

```
do
```

```
{
```

```
    printf("Enter the number of elements: ");
```

```
    scanf("%d", &n);
```

```
    if (n <= 0 || n > MAX)
```

```
    {
```

```
        printf("Invalid size. Enter a value between 1 and %d.\n", MAX);
```

```
    }
```

```
} while (n <= 0 || n > MAX);
```

```
/* Read array */
```

```
printf("Enter %d array elements:\n", n);
```

```
for (i = 0; i < n; i++)
```

```
{
```

```
    scanf("%d", &a[i]);
```

```
}
```

```
do
```

```

{
    printf("\n===== MENU =====\n");
    printf("1. Bubble Sort and Binary Search\n");
    printf("2. Insertion and Deletion\n");
    printf("3. Right Rotation\n");
    printf("4. Exit\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            /* Bubble Sort */
            for (i = 0; i < n - 1; i++)
            {
                for (j = 0; j < n - i - 1; j++)
                {
                    if (a[j] > a[j + 1])
                    {
                        temp = a[j];
                        a[j] = a[j + 1];
                        a[j + 1] = temp;
                    }
                }
            }

            printf("\nSorted array:\n");

```

```
for (i = 0; i < n; i++)  
{  
    printf("%d ", a[i]);  
}
```

```
printf("\n");
```

```
/* Binary Search */
```

```
printf("Enter the element to search: ");
```

```
scanf("%d", &key);
```

```
low = 0;
```

```
high = n - 1;
```

```
found = 0;
```

```
while (low <= high)
```

```
{  
    mid = (low + high) / 2;
```

```
    if (a[mid] == key)
```

```
    {  
        found = 1;  
        break;
```

```
    }  
    else if (key < a[mid])
```

```
    {  
        high = mid - 1;
```

```
    }  
    else  
    {  
        low = mid + 1;  
    }  
}  
  
if (found)  
{  
    printf("Element %d found at position %d.\n",  
        key, mid + 1);  
}  
else  
{  
    printf("Element %d not found.\n", key);  
}  
  
break;
```

case 2:

```
    printf("\n1. Insertion\n");  
    printf("2. Deletion\n");  
  
    printf("Enter your choice: ");  
    scanf("%d", &subChoice);  
  
    switch (subChoice)  
    {
```

case 1:

```
if (n == MAX)
{
    printf("Array is full. Insertion not possible.\n");
    break;
}

printf("Enter the position for insertion (1 to %d): ",
    n + 1);
scanf("%d", &position);

if (position < 1 || position > n + 1)
{
    printf("Invalid position.\n");
    break;
}

printf("Enter the element to insert: ");
scanf("%d", &element);

for (i = n; i >= position; i--)
{
    a[i] = a[i - 1];
}

a[position - 1] = element;
n++;
```



```
printf("Array after insertion:\n");
```

```
for (i = 0; i < n; i++)  
{  
    printf("%d ", a[i]);  
}
```

```
printf("\n");  
break;
```

case 2:

```
if (n == 0)  
{  
    printf("Array is empty. Deletion not possible.\n");  
    break;  
}
```

```
printf("Enter the position for deletion (1 to %d): ",  
       n);  
scanf("%d", &position);
```

```
if (position < 1 || position > n)  
{  
    printf("Invalid position.\n");  
    break;  
}
```

```
element = a[position - 1];
```

```
for (i = position - 1; i < n - 1; i++)
```

```
{
```

```
    a[i] = a[i + 1];
```

```
}
```

```
n--;
```

```
printf("Deleted element: %d\n", element);
```

```
printf("Array after deletion:\n");
```

```
for (i = 0; i < n; i++)
```

```
{
```

```
    printf("%d ", a[i]);
```

```
}
```

```
printf("\n");
```

```
break;
```

```
default:
```

```
    printf("Invalid insertion/deletion choice.\n");
```

```
}
```

```
break;
```

```
case 3:
```

```
    if (n == 0)
```

```
{  
    printf("Array is empty. Rotation not possible.\n");  
    break;  
}
```

```
printf("Enter the number of right rotations (k): ");  
scanf("%d", &k);
```

```
if (k < 0)  
{  
    printf("Invalid value of k.\n");  
    break;  
}
```

```
k = k % n;
```

```
for (r = 0; r < k; r++)
```

```
{  
    last = a[n - 1];
```

```
    for (i = n - 1; i > 0; i--)
```

```
    {  
        a[i] = a[i - 1];  
    }
```

```
    a[0] = last;
```

```
}
```

```
printf("Array after right rotation by %d position(s):\n", k);
```

```
for (i = 0; i < n; i++)
```

```
{
```

```
    printf("%d ", a[i]);
```

```
}
```

```
printf("\n");
```

```
break;
```

```
case 4:
```

```
    printf("Exiting the program...\n");
```

```
    break;
```

```
default:
```

```
    printf("Invalid menu choice. Please try again.\n");
```

```
}
```

```
} while (choice != 4);
```

```
return 0;
```

```
}
```

❖ Test case table

Test	Input / Operation	Expected Result
1	Array: 40 10 30 20 50, Choice 1, Search 30	Sorted: 10 20 30 40 50; 30 found at position 3
2	Array: 8 2 6 4, Choice 1, Search 5	Sorted: 2 4 6 8; element not found
3	Array: 10 20 30 40, insert 25 at position 3	10 20 25 30 40
4	Array: 10 20 30 40, insert 5 at position 1	5 10 20 30 40
5	Array: 10 20 30 40, delete position 2	Deleted 20; array becomes 10 30 40
6	Array: 10 20 30 40, delete position 4	Deleted 40; array becomes 10 20 30
7	Array: 1 2 3 4 5, k = 2	4 5 1 2 3
8	Array: 1 2 3 4 5, k = 7	4 5 1 2 3, since $7 \% 5 = 2$
9	Insertion at position 0	Invalid position
10	Deletion at position greater than n	Invalid position
11	k = 0	Array remains unchanged
12	Main-menu choice 8	Invalid menu choice
13	Choice 4	Program terminates

❖ Compile, Execute and Evaluate

```
(kali㉿kali)-[~/Desktop/exp30]  
$ vi array3.c
```

```
(kali㉿kali)-[~/Desktop/exp30]  
$ gcc array3.c -o array3
```

```
(kali㉿kali)-[~/Desktop/exp30]  
$ ./array3
```

Enter the number of elements: 5

Enter 5 array elements:

40 10 30 20 50

===== MENU =====

1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation
4. Exit

Enter your choice: 1

Sorted array:

10 20 30 40 50

Enter the element to search: 30

Element 30 found at position 3.

===== MENU =====

1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation
4. Exit

Enter your choice: 4

Exiting the program...

```
(kali㉿kali)-[~/Desktop/exp30]  
$ ./array3
```

Enter the number of elements: 4

Enter 4 array elements:

8 2 6 4

===== MENU =====

===== MENU =====

1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation
4. Exit

Enter your choice: 1

Sorted array:

2 4 6 8

Enter the element to search: 5

Element 5 not found.

===== MENU =====

1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation
4. Exit

Enter your choice: 4

Exiting the program...

(kali@kali)-[~/Desktop/exp30]

\$./array3

Enter the number of elements: 4

Enter 4 array elements:

10 20 30 40

===== MENU =====

1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation
4. Exit

Enter your choice: 2

1. Insertion

2. Deletion

Enter your choice: 1

Enter the position for insertion (1 to 5): 3

Enter the element to insert: 25

Array after insertion:

Array after insertion:

10 20 25 30 40

===== MENU =====

1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation
4. Exit

Enter your choice: 2

1. Insertion
2. Deletion

Enter your choice: 1

Enter the position for insertion (1 to 6): 1

Enter the element to insert: 5

Array after insertion:

5 10 20 25 30 40

===== MENU =====

1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation
4. Exit

Enter your choice: 2

1. Insertion
2. Deletion

Enter your choice: 2

Enter the position for deletion (1 to 6): 2

Deleted element: 10

Array after deletion:

5 20 25 30 40

===== MENU =====

1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation
4. Exit

Enter your choice: 2


```
1. Insertion
2. Deletion
Enter your choice: 2
Enter the position for deletion (1 to 5): 4
Deleted element: 30
Array after deletion:
5 20 25 40
```

```
===== MENU =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation
4. Exit
Enter your choice: 4
Exiting the program...
```

```
(kali㉿kali)-[~/Desktop/exp30]
$ ./array3
Enter the number of elements: 5
Enter 5 array elements:
1 2 3 4 5
```

```
===== MENU =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation
4. Exit
Enter your choice: 3
Enter the number of right rotations (k): 2
Array after right rotation by 2 position(s):
4 5 1 2 3
```

```
===== MENU =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation
4. Exit
Enter your choice: 3
```

```
Enter your choice: 3
Enter the number of right rotations (k): 7
Array after right rotation by 2 position(s):
2 3 4 5 1
```

```
===== MENU =====
```

1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation
4. Exit

```
Enter your choice: 4
Exiting the program...
```

```
(kali@kali)-[~/Desktop/exp30]
```

```
$ ./array3
```

```
Enter the number of elements: 1
```

```
Enter 1 array elements:
```

```
1
```

```
===== MENU =====
```

1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation
4. Exit

```
Enter your choice: 2
```

```
1. Insertion
```

```
2. Deletion
```

```
Enter your choice: 1
```

```
Enter the position for insertion (1 to 2): 0
```

```
Invalid position.
```

```
===== MENU =====
```

1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation
4. Exit

```
Enter your choice: 2
```

```
1. Insertion
2. Deletion
Enter your choice: 2
Enter the position for deletion (1 to 1): 2
Invalid position.
```

```
===== MENU =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation
4. Exit
Enter your choice: 3
Enter the number of right rotations (k): 0
Array after right rotation by 0 position(s):
1
```

```
===== MENU =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation
4. Exit
Enter your choice: 8
Invalid menu choice. Please try again.
```

```
===== MENU =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation
4. Exit
Enter your choice: 4
Exiting the program...
```

```
(kali㉿kali)-[~/Desktop/exp30]
$
```